COMSATS University Islamabad

Attock Campus

Department Of Computer Science

Semester Project

Course	***Database Systems***
Instructor	***Sir Shahzad Faisal***

Students Details

Registration No.	***Name***
FA24-BAI-028	***Husna Zia***
FA24-BAI-055	***Zarish Batool***
FA24-BAI-053	***Zeenat Bibi***

Project Title: **“Inventory management system”**

Project Description:

The *Inventory Management System (INVEN)* is a database-backed application that handles inventory operations such as product management, purchase order processing, supply order processing, and report generation. The system tracks products, customers, suppliers, and orders to ensure accurate stock levels, order fulfillment, and reporting. It is designed to reduce data redundancy and enforce data integrity through normalization of its database structure.

Entities, Attributes and Keys:

Entity Name	Attributes	Primary Key (PK)	Foreign Key (FK)
Admin	AdminID, AdminName, Username, Password, Role	**AdminID**	–
Customer	CustomerID, CustomerName, Phone, Email, Address	**CustomerID**	–
Supplier	SupplierID, SupplierName, Phone, Email, Address	**SupplierID**	–
Product	ProductID, ProductName, Category, Price, QuantityInStock, AdminID	**ProductID**	AdminID → Admin(AdminID)
PurchaseOrder	PurchaseOrderID, CustomerID, OrderDate, TotalAmount	**PurchaseOrderID**	CustomerID → Customer(CustomerID)
SupplyOrder	SupplyOrderID, SupplierID, OrderDate, TotalCost	**SupplyOrderID**	SupplierID → Supplier(SupplierID)
PurchaseOrderDetail	PurchaseOrderID, ProductID, Quantity,	**PurchaseOrderID + ProductID** (composite)	PurchaseOrderID → PurchaseOrder(PurchaseOrderID), ProductID →

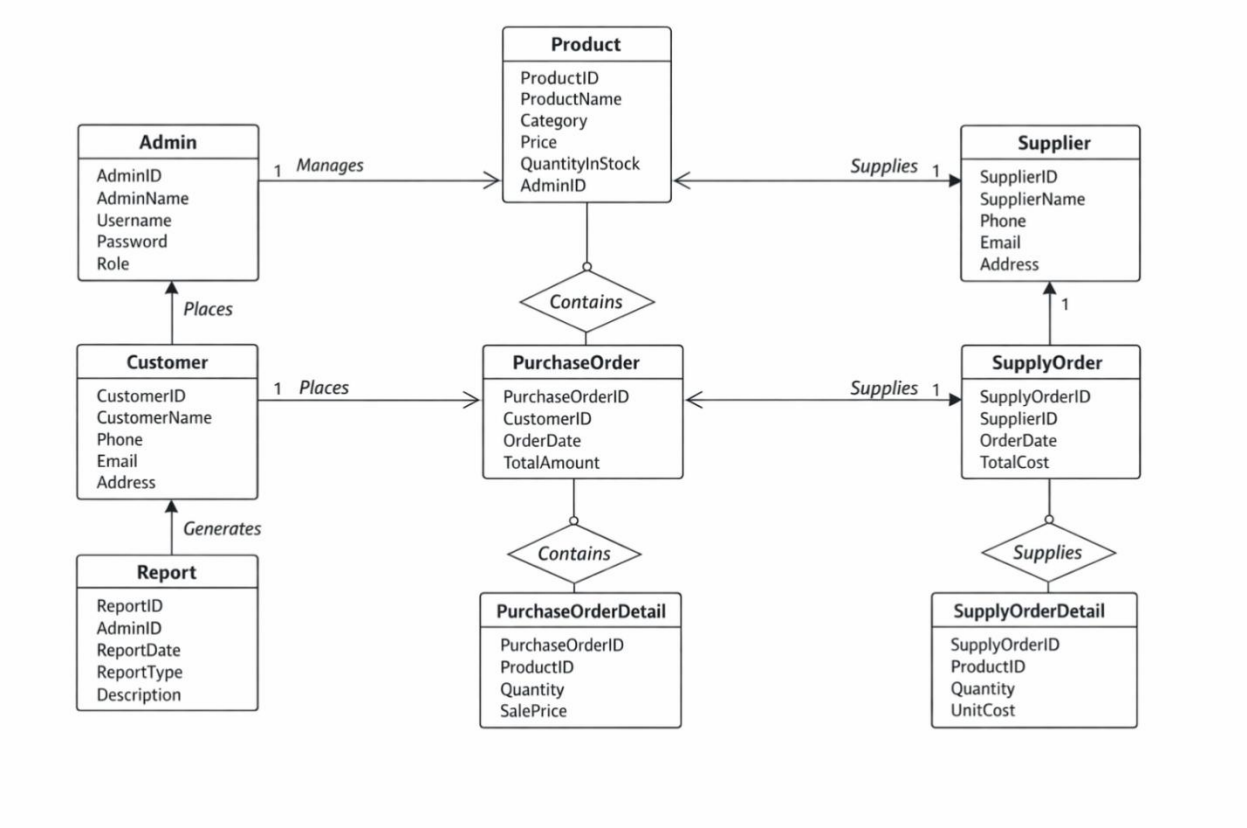
	SalePrice		Product(ProductID)
SupplyOrderDetail	SupplyOrderID, ProductID, Quantity, UnitCost	**SupplyOrderID + ProductID** (composite)	SupplyOrderID → SupplyOrder(SupplyOrderID), ProductID → Product(ProductID)
Report	ReportID, AdminID, ReportDate, ReportType, Description	**ReportID**	AdminID → Admin(AdminID)

Relationships & Cardinality Constraints:

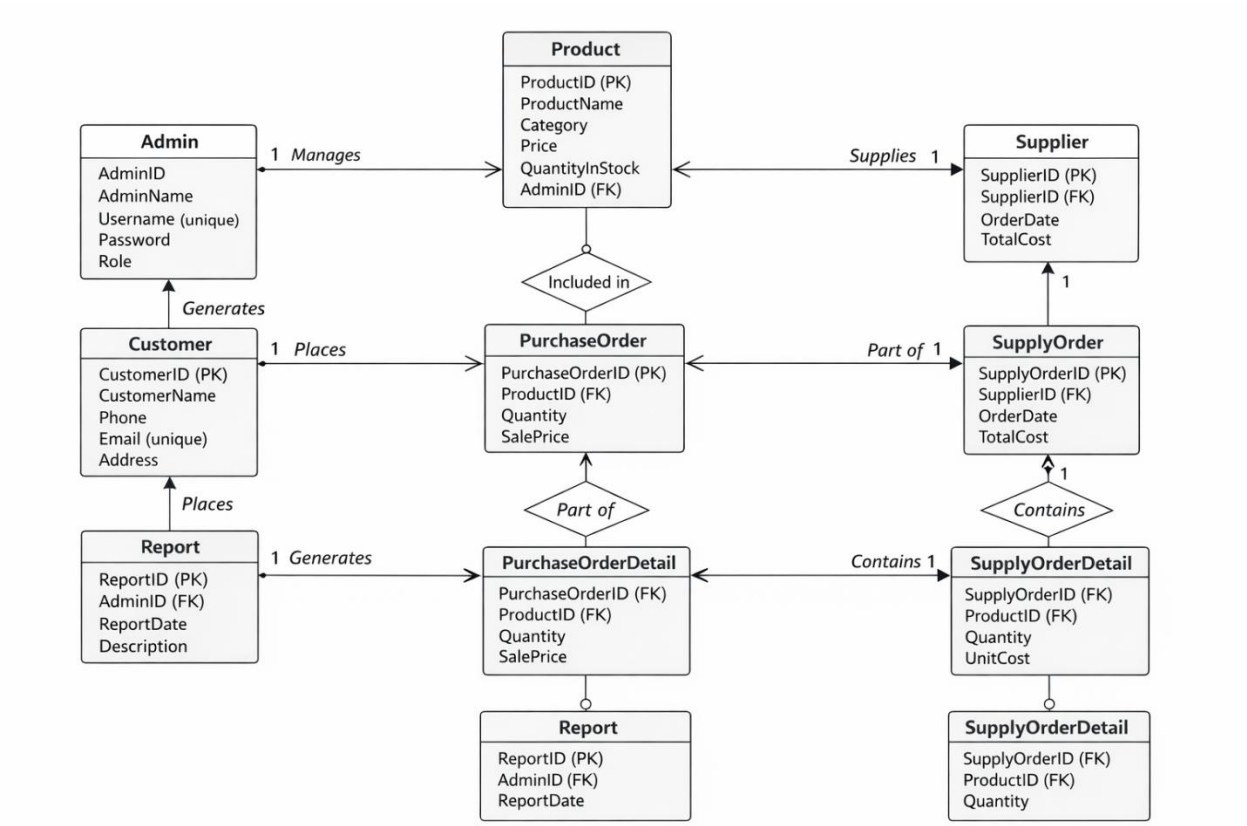
Relationship	From Entity	To Entity	Type	Cardinality (From → To)	Description
Admin manages Product	Admin	Product	One-to-Many	1 → Many	Each admin can manage multiple products.
Admin generates Report	Admin	Report	One-to-Many	1 → Many	Admin can create multiple reports.
Customer places PurchaseOrder	Customer	PurchaseOrder	One-to-Many	1 → Many	Each customer can place multiple purchase orders.
Supplier supplies SupplyOrder	Supplier	SupplyOrder	One-to-Many	1 → Many	Each supplier can supply multiple orders.
PurchaseOrder contains Product (via PurchaseOrderDetail)	PurchaseOrder	Product	Many-to-Many	Many → Many	Each purchase order can contain multiple products; each product can be in multiple

					orders.
SupplyOrder contains Product (via SupplyOrderDetail)	SupplyOrder	Product	Many-to-Many	Many → Many	Each supply order can contain multiple products; each product can be in multiple supply orders.

ERD :

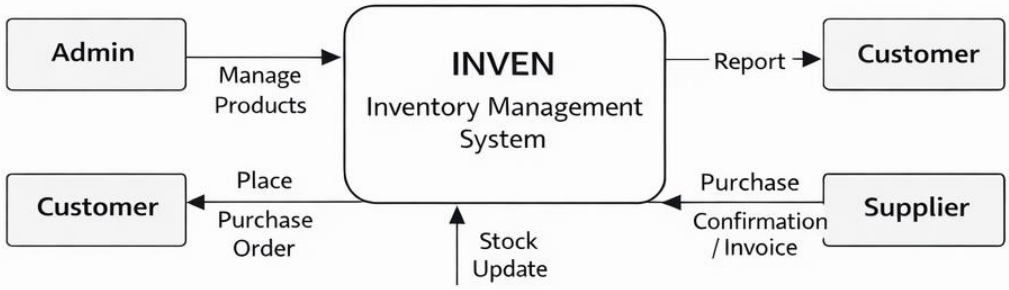


EERD:

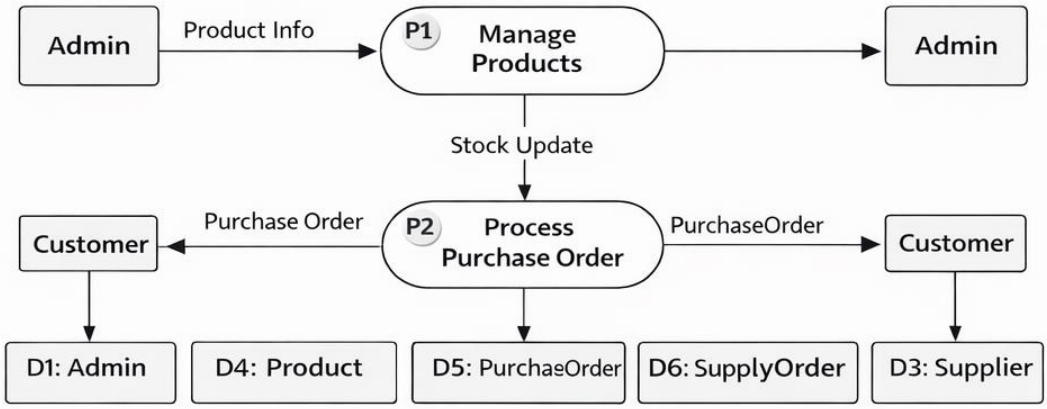

Product
ProductID (PK)
ProductName
Category
Price
QuantityInStock
AdminID (FK)
Admin
AdminID
AdminName
Username (unique)
Password
Role
1 Manages
Supplies 1
Supplier
SupplierID (PK)
SupplierID (FK)
OrderDate
TotalCost
Included in
1
Generates
Customer
CustomerID (PK)
CustomerName
Phone
Email (unique)
Address
1 Places
PurchaseOrder
PurchaseOrderID (PK)
ProductID (FK)
Quantity
SalePrice
Part of 1
SupplyOrder
SupplyOrderID (PK)
SupplierID (FK)
OrderDate
TotalCost
1
Places
Part of
Contains
1
Report
ReportID (PK)
AdminID (FK)
ReportDate
Description
1 Generates
PurchaseOrderDetail
PurchaseOrderID (FK)
ProductID (FK)
Quantity
SalePrice
Contains 1
SupplyOrderDetail
SupplyOrderID (FK)
ProductID (FK)
Quantity
UnitCost
Report
ReportID (PK)
AdminID (FK)
ReportDate
SupplyOrderDetail
SupplyOrderID (FK)
ProductID (FK)
Quantity

DFD:

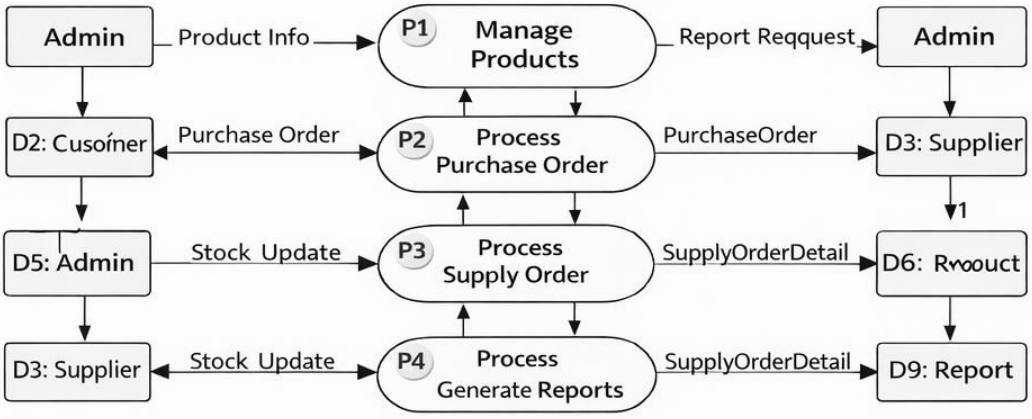
DFD Level 0 (Context Diagram)



DFD Level 1 (Detailed Processes)



DFD Level 1 (Detailed Processes)



Unnormalized Form (UNF)

Inventory_UNF – single large table combining admins, customers, suppliers, products, purchase orders, supply orders, and reports.

1. Admin + Products (with repeating products per admin)

AdminID	AdminName	Username	Role	ProductID	ProductName	Category	Price	QuantityInStock
A001	Alex	alex01	Manager	P001	Laptop	Electronics	1000	50
A001	Alex	alex01	Manager	P002	Mouse	Electronics	20	200
A002	Sara	sara02	Supervisor	P003	Keyboard	Electronics	30	150

---

2. Customers + PurchaseOrders + Products (repeating products per order)

CustomerID	Customer Name	Phone	Email	Address	PurchaseOrderID	Order Date	ProductID	Quantity	SalePrice
C001	John Doe	6666666666	john@email.com	City A	PO001	2025-12-01	P001	2	2000
C001	John Doe	6666666666	john@email.com	City A	PO001	2025-12-01	P002	5	100
C002	Jane Smith	5555555555	jane@email.com	City B	PO002	2025-12-05	P003	1	30

---

3. Suppliers + SupplyOrders + Products (repeating products per supply)

SupplierID	SupplierName	Phone	Email	Address	SupplyOrderID	OrderDate	ProductID	Quantity	UnitCost
S001	Supplier1	7777777777	supplier1@email.com	City X	SO001	2025-11-20	P001	10	900
S001	Supplier1	7777777777	supplier1@email.com	City X	SO001	2025-11-20	P002	15	15
S002	Supplier2	88888888	supplier2@email.com	City Y	SO002	2025-11-25	P003	20	25

---

4. Reports (Admin generates multiple reports)

ReportID	AdminID	AdminName	ReportDate	ReportType	Description
R001	A001	Alex	2025-12-05	Inventory	Monthly inventory report
R002	A001	Alex	2025-12-10	Sales	Weekly sales report
R003	A002	Sara	2025-12-07	Purchase	Supplier purchase report

Issues in UNF:

- Multiple entities in one table
- Redundant data (Admin, Customer, Supplier repeated)
- Repeating order/product info
- Update/Insert/Delete anomalies

2 First Normal Form (1NF)

Split repeating groups and make values atomic:

1. **Admin**

AdminID	AdminName	Username	Role
A001	Alex	alex01	Manager

2. **Customer**

CustomerID	CustomerName	Phone	Email	Address
C001	John Doe	6666666666	john@email.com	City A
C002	Jane Smith	5555555555	jane@email.com	City B

3. **Supplier**

SupplierID	SupplierName	Phone	Email	Address
S001	Supplier1	7777777777	supplier1@email.com	City X
S002	Supplier2	8888888888	supplier2@email.com	City Y

4. **Product**

ProductID	ProductName	Category	Price	QuantityInStock	AdminID
P001	Product1	Cat1	100	50	A001
P002	Product2	Cat2	200	30	A001

5. **PurchaseOrderDetail**

PurchaseOrderID	ProductID	Quantity	SalePrice
PO001	P001	5	500
PO001	P002	2	400

6. **SupplyOrderDetail**

SupplyOrderID	ProductID	Quantity	UnitCost
SO001	P001	10	90
SO001	P002	15	180

7. **PurchaseOrder**

PurchaseOrderID	CustomerID	OrderDate	TotalAmount
PO001	C001	2025-12-01	900

8. **SupplyOrder**

SupplyOrderID	SupplierID	OrderDate	TotalCost
SO001	S001	2025-12-02	2700

9. **Report**

ReportID	AdminID	ReportDate	ReportType	Description
R001	A001	2025-12-05	Inventory	Monthly report

3 Second Normal Form (2NF)

Remove partial dependencies (non-key attributes depend on full PK).

Tables already in 1NF with single attribute PK are in 2NF:

- Admin, Customer, Supplier, Product, PurchaseOrder, SupplyOrder, Report

Composite PK tables (junction tables) are also in 2NF:

- PurchaseOrderDetail: Quantity, SalePrice depend on both PurchaseOrderID + ProductID
- SupplyOrderDetail: Quantity, UnitCost depend on both SupplyOrderID + ProductID

All tables are in **2NF**.

4 Third Normal Form (3NF)

Remove transitive dependencies.

1. Product table: AdminID is FK, but all other attributes depend only on ProductID → OK
2. PurchaseOrder: CustomerID FK, other attributes depend only on PurchaseOrderID → OK
3. SupplyOrder: SupplierID FK, other attributes depend only on SupplyOrderID → OK
4. Reports: AdminID FK, other attributes depend only on ReportID → OK

No transitive dependencies remain → fully normalized.

---

5 Final Normalized Tables (3NF)

Table Name	Primary Key	Foreign Key	Attributes
Admin	AdminID	–	AdminName, Username, Role
Customer	CustomerID	–	CustomerName, Phone, Email, Address
Supplier	SupplierID	–	SupplierName, Phone, Email, Address
Product	ProductID	AdminID → Admin	ProductName, Category, Price, QuantityInStock
PurchaseOrder	PurchaseOrderID	CustomerID → Customer	OrderDate, TotalAmount
SupplyOrder	SupplyOrderID	SupplierID → Supplier	OrderDate, TotalCost
PurchaseOrderDetail	PurchaseOrderID + ProductID	PurchaseOrderID → PurchaseOrder, ProductID → Product	Quantity, SalePrice
SupplyOrderDetail	SupplyOrderID + ProductID	SupplyOrderID → SupplyOrder, ProductID → Product	Quantity, UnitCost
Report	ReportID	AdminID → Admin	ReportDate, ReportType, Description

MYSQL CODE

```
CREATE DATABASE INVEN;

USE  INVEN;

-- DDL: Create Tables and Define Relationships

-- Drop tables in reverse order for a clean setup

DROP TABLE IF EXISTS Report;

DROP VIEW IF EXISTS High_Value_Customer_View;

DROP VIEW IF EXISTS Admin_Activity_Report_View;

DROP VIEW IF EXISTS Supplier_Delivery_Report_View;

DROP VIEW IF EXISTS Current_Inventory_Status_View;

DROP VIEW IF EXISTS Purchase_Summary_View;

DROP PROCEDURE IF EXISTS Place_Purchase_Order;

DROP FUNCTION IF EXISTS Get_Reorder_Status;

DROP TRIGGER IF EXISTS trg_Purchase_Stock_OUT;

DROP TRIGGER IF EXISTS trg_Supply_Stock_IN;

DROP TRIGGER IF EXISTS trg_Update_PurchaseOrder_Total;

DROP TRIGGER IF EXISTS trg_Update_SupplyOrder_Total;


DROP TABLE IF EXISTS PurchaseOrderDetail;

DROP TABLE IF EXISTS SupplyOrderDetail;

DROP TABLE IF EXISTS PurchaseOrder;

DROP TABLE IF EXISTS SupplyOrder;

DROP TABLE IF EXISTS Product;

DROP TABLE IF EXISTS Customer;

DROP TABLE IF EXISTS Supplier;

DROP TABLE IF EXISTS Admin;
```

```
-- Admin Table
CREATE TABLE Admin (
   AdminID INT PRIMARY KEY,
   AdminName VARCHAR(100) NOT NULL,
   Username VARCHAR(50) UNIQUE NOT NULL,
   Password VARCHAR(50) NOT NULL,
   Role VARCHAR(50)
   );
-- Customer Table
CREATE TABLE Customer (
   CustomerID INT PRIMARY KEY,
   CustomerName VARCHAR(100) NOT NULL,
   Phone VARCHAR(20),
   Email VARCHAR(100) UNIQUE,
   Address VARCHAR(255)
);
-- Supplier Table
CREATE TABLE Supplier (
   SupplierID INT PRIMARY KEY,
   SupplierName VARCHAR(100) NOT NULL,
   Phone VARCHAR(20),
   Email VARCHAR(100) UNIQUE,
   Address VARCHAR(255)
);
-- Product Table (Includes FK to AdminID)
CREATE TABLE Product (
   ProductID INT PRIMARY KEY,
```

```
    ProductName VARCHAR(150) NOT NULL,
    Category VARCHAR(100),
    Price DECIMAL(10, 2) NOT NULL,
    QuantityInStock INT DEFAULT 0,
    AdminID INT,
    FOREIGN KEY (AdminID) REFERENCES Admin(AdminID)
);
-- PurchaseOrder Table
CREATE TABLE PurchaseOrder (
    PurchaseOrderID INT PRIMARY KEY,
    CustomerID INT,
    OrderDate DATE NOT NULL,
    TotalAmount DECIMAL(10, 2) DEFAULT 0.00,
    FOREIGN KEY (CustomerID) REFERENCES Customer(CustomerID)
);
-- SupplyOrder Table
CREATE TABLE SupplyOrder (
    SupplyOrderID INT PRIMARY KEY,
    SupplierID INT,
    OrderDate DATE NOT NULL,
    TotalCost DECIMAL(10, 2) DEFAULT 0.00,
    FOREIGN KEY (SupplierID) REFERENCES Supplier(SupplierID)
);
-- Report Table
CREATE TABLE Report (
    ReportID INT PRIMARY KEY,
    AdminID INT,
```

```
    ReportDate DATE NOT NULL,
    ReportType VARCHAR(50),
    Description LONGTEXT, -- Corrected to LONGTEXT for MySQL compatibility
    FOREIGN KEY (AdminID) REFERENCES Admin(AdminID)
);
-- Junction Table for PurchaseOrder (Many-to-Many)
CREATE TABLE PurchaseOrderDetail (
    PurchaseOrderID INT NOT NULL,
    ProductID INT NOT NULL,
    Quantity INT NOT NULL,
    SalePrice DECIMAL(10, 2) NOT NULL,
    PRIMARY KEY (PurchaseOrderID, ProductID),
    FOREIGN KEY (PurchaseOrderID) REFERENCES PurchaseOrder(PurchaseOrderID) ON DELETE
CASCADE,
    FOREIGN KEY (ProductID) REFERENCES Product(ProductID)
);
-- Junction Table for SupplyOrder (Many-to-Many)
CREATE TABLE SupplyOrderDetail (
    SupplyOrderID INT NOT NULL,
    ProductID INT NOT NULL,
    Quantity INT NOT NULL,
    UnitCost DECIMAL(10, 2) NOT NULL,
    PRIMARY KEY (SupplyOrderID, ProductID),
    FOREIGN KEY (SupplyOrderID) REFERENCES SupplyOrder(SupplyOrderID) ON DELETE
CASCADE,
    FOREIGN KEY (ProductID) REFERENCES Product(ProductID)
);
```

-- DML: Insert Sample Data

INSERT INTO Admin VALUES (1, 'Ayesha Ahmad', 'ayesha_admin', 'admin123', 'Manager');

INSERT INTO Admin VALUES (2, 'Zain Raza', 'zain_admin', 'admin456', 'Supervisor');

INSERT INTO Admin VALUES (3, 'Faizan Malik', 'faizan_admin', 'admin789', 'Data Analyst');

INSERT INTO Admin VALUES (4, 'Mehwish Noor', 'mehwish_admin', 'mehwish321', 'Inventory Manager');

INSERT INTO Customer VALUES (1, 'Ali Khan', '03123456789', 'ali@gmail.com', 'Karachi');

INSERT INTO Customer VALUES (2, 'Sara Iqbal', '03214567890', 'sara@gmail.com', 'Lahore');

INSERT INTO Customer VALUES (3, 'Bilal Ahmad', '03335678901', 'bilal@yahoo.com', 'Islamabad');

INSERT INTO Customer VALUES (4, 'Nida Farooq', '03001234567', 'nida.farooq@gmail.com', 'Rawalpindi');

INSERT INTO Customer VALUES (5, 'Usman Tariq', '03451112223', 'usman@yahoo.com', 'Peshawar');

INSERT INTO Customer VALUES (6, 'Hira Sajjad', '03111223344', 'hira@live.com', 'Multan');

INSERT INTO Supplier VALUES (1, 'Tech Supply Co.', '03211234567', 'supply@tech.com', 'Lahore');

INSERT INTO Supplier VALUES (2, 'Office Solutions', '03456789012', 'office@supplies.com', 'Faisalabad');

INSERT INTO Supplier VALUES (3, 'Pak IT Distributors', '03118889900', 'sales@pakit.com', 'Quetta');

INSERT INTO Supplier VALUES (4, 'Galaxy Supplies', '03229997766', 'contact@galaxysupplies.pk', 'Sialkot');

INSERT INTO Product (ProductID, ProductName, Category, Price, QuantityInStock, AdminID)

VALUES

(101, 'Wireless Mouse', 'Electronics', 1200.00, 150, 4),

(102, 'Mechanical Keyboard', 'Electronics', 3500.00, 100, 4),

(103, 'Laptop Stand', 'Accessories', 2200.00, 75, 4),

(104, 'USB-C Hub', 'Accessories', 1800.00, 90, 4),

(105, 'Webcam HD', 'Electronics', 3000.00, 60, 4),

(106, 'Bluetooth Speaker', 'Audio', 4500.00, 40, 4),

(107, 'LED Monitor 24"', 'Displays', 17000.00, 30, 4);

-- TotalAmount will be updated by Triggers (Initial value 0.00)

INSERT INTO PurchaseOrder (PurchaseOrderID, CustomerID, OrderDate) VALUES

(201, 1, '2024-06-01'),

(202, 2, '2024-06-05'),

(203, 3, '2024-06-08'),

(204, 4, '2024-06-10'),

(205, 5, '2024-06-11'),

(206, 6, '2024-06-12');

-- Inserting details triggers stock decrease and TotalAmount calculation

INSERT INTO PurchaseOrderDetail VALUES (201, 101, 2, 1200.00);

INSERT INTO PurchaseOrderDetail VALUES (202, 102, 1, 3500.00);

INSERT INTO PurchaseOrderDetail VALUES (203, 103, 1, 2200.00);

INSERT INTO PurchaseOrderDetail VALUES (203, 104, 1, 1800.00);

INSERT INTO PurchaseOrderDetail VALUES (204, 105, 1, 3000.00);

INSERT INTO PurchaseOrderDetail VALUES (204, 106, 1, 4500.00);

INSERT INTO PurchaseOrderDetail VALUES (205, 107, 1, 12000.00);

INSERT INTO PurchaseOrderDetail VALUES (206, 101, 1, 1000.00);

INSERT INTO PurchaseOrderDetail VALUES (206, 103, 1, 1000.00);

-- TotalCost will be updated by Triggers (Initial value 0.00)

INSERT INTO SupplyOrder (SupplyOrderID, SupplierID, OrderDate) VALUES

(301, 1, '2024-05-25'),

(302, 2, '2024-06-02'),

```
(303, 3, '2024-06-03'),
(304, 4, '2024-06-04');
-- Inserting details triggers stock increase and TotalCost calculation
INSERT INTO SupplyOrderDetail VALUES (301, 101, 50, 500.00);
INSERT INTO SupplyOrderDetail VALUES (301, 102, 50, 700.00);
INSERT INTO SupplyOrderDetail VALUES (302, 103, 20, 1500.00);
INSERT INTO SupplyOrderDetail VALUES (302, 104, 10, 1500.00);
INSERT INTO SupplyOrderDetail VALUES (303, 105, 20, 2000.00);
INSERT INTO SupplyOrderDetail VALUES (303, 106, 10, 1500.00);
INSERT INTO SupplyOrderDetail VALUES (304, 107, 2, 15000.00);

INSERT INTO Report VALUES (401, 1, '2024-06-01', 'Purchase', 'Purchase order report for June 1st');
INSERT INTO Report VALUES (402, 2, '2024-06-05', 'Supply', 'Supply order from Office Solutions on June 2nd');
INSERT INTO Report VALUES (403, 3, '2024-06-10', 'Purchase', 'Customer purchase log generated by Faizan');
INSERT INTO Report VALUES (404, 4, '2024-06-12', 'Supply', 'June supply overview generated by Mehwish');
-- TRIGGERS: Automated stock and total updates
DELIMITER //
-- 1. Decrease Stock on New Purchase Order Detail
CREATE TRIGGER trg_Purchase_Stock_OUT
AFTER INSERT ON PurchaseOrderDetail
FOR EACH ROW
BEGIN
    UPDATE Product
    SET QuantityInStock = QuantityInStock - NEW.Quantity
```

```
    WHERE ProductID = NEW.ProductID;
END //
-- 2. Increase Stock on New Supply Order Detail
CREATE TRIGGER trg_Supply_Stock_IN
AFTER INSERT ON SupplyOrderDetail
FOR EACH ROW
BEGIN
    UPDATE Product
    SET QuantityInStock = QuantityInStock + NEW.Quantity
    WHERE ProductID = NEW.ProductID;
END //
-- 3. Update PurchaseOrder Total Amount (ensures data in one relation updates data in others)
CREATE TRIGGER trg_Update_PurchaseOrder_Total
AFTER INSERT ON PurchaseOrderDetail
FOR EACH ROW
BEGIN
    UPDATE PurchaseOrder
    SET TotalAmount = (
        SELECT SUM(Quantity * SalePrice)
        FROM PurchaseOrderDetail
        WHERE PurchaseOrderID = NEW.PurchaseOrderID )
    WHERE PurchaseOrderID = NEW.PurchaseOrderID;
END //
-- 4. Update SupplyOrder Total Cost (ensures data in one relation updates data in others)
CREATE TRIGGER trg_Update_SupplyOrder_Total
AFTER INSERT ON SupplyOrderDetail
```

```
FOR EACH ROW
BEGIN
   UPDATE SupplyOrder
   SET TotalCost = (
      SELECT SUM(Quantity * UnitCost)
      FROM SupplyOrderDetail
      WHERE SupplyOrderID = NEW.SupplyOrderID  )
   WHERE SupplyOrderID = NEW.SupplyOrderID;
END //
DELIMITER ;
-- STORED PROCEDURE: Handle Order Placement Transaction (with stock check)
DELIMITER //
CREATE PROCEDURE Place_Purchase_Order (
   IN p_PurchaseOrderID INT,
   IN p_CustomerID INT,
   IN p_OrderDate DATE,
   IN p_ProductID INT,
   IN p_Quantity INT)
BEGIN
   DECLARE v_Price DECIMAL(10, 2);
   DECLARE v_Stock INT;
   SELECT Price, QuantityInStock INTO v_Price, v_Stock
   FROM Product
   WHERE ProductID = p_ProductID;
   -- Check for sufficient stock
   IF v_Stock >= p_Quantity THEN
      -- 1. Insert or ignore into main PurchaseOrder (if it doesn't exist)
```

```
        INSERT IGNORE INTO PurchaseOrder (PurchaseOrderID, CustomerID, OrderDate,
TotalAmount)
        VALUES (p_PurchaseOrderID, p_CustomerID, p_OrderDate, 0.00);
        -- 2. Insert into PurchaseOrderDetail (Triggers will handle stock and total update)
        INSERT INTO PurchaseOrderDetail (PurchaseOrderID, ProductID, Quantity, SalePrice)
        VALUES (p_PurchaseOrderID, p_ProductID, p_Quantity, v_Price);
        SELECT 'SUCCESS: Order placed and inventory updated.' AS Status;
    ELSE
        SELECT CONCAT('FAILED: Insufficient stock for ProductID ', p_ProductID, '. Available: ',
v_Stock) AS Status;
    END IF;
END //
DELIMITER ;

-- VIEWS: Simplify Reporting and Data Access
-- V1: Purchase_Summary_View
CREATE VIEW Purchase_Summary_View AS
SELECT
    PO.PurchaseOrderID,
    PO.OrderDate,
    C.CustomerName,
    PO.TotalAmount,
    COUNT(POD.ProductID) AS TotalItems,
    A.AdminName AS InventoryManager
FROM
    PurchaseOrder PO
JOIN
    Customer C ON PO.CustomerID = C.CustomerID
```

```
LEFT JOIN
    PurchaseOrderDetail POD ON PO.PurchaseOrderID = POD.PurchaseOrderID
LEFT JOIN
    Product P ON POD.ProductID = P.ProductID
LEFT JOIN
    Admin A ON P.AdminID = A.AdminID
GROUP BY
    PO.PurchaseOrderID, PO.OrderDate, C.CustomerName, PO.TotalAmount, A.AdminName;

-- V2: Current_Inventory_Status_View (Uses the custom function)
CREATE VIEW Current_Inventory_Status_View AS
SELECT
    P.ProductID,
    P.ProductName,
    P.Category,
    P.QuantityInStock,
    P.Price,
    Get_Reorder_Status(P.ProductID) AS InventoryStatus
FROM
    Product P;
-- V3: Supplier_Delivery_Report_View
CREATE VIEW Supplier_Delivery_Report_View AS
SELECT
    SO.SupplyOrderID,
    SO.OrderDate AS DeliveryDate,
    S.SupplierName,
    SO.TotalCost,
```

```
    COUNT(SOD.ProductID) AS UniqueProductsSupplied
FROM
    SupplyOrder SO
JOIN
    Supplier S ON SO.SupplierID = S.SupplierID
LEFT JOIN
    SupplyOrderDetail SOD ON SO.SupplyOrderID = SOD.SupplyOrderID
GROUP BY
    SO.SupplyOrderID, SO.OrderDate, S.SupplierName, SO.TotalCost;
-- V4: Admin_Activity_Report_View
CREATE VIEW Admin_Activity_Report_View AS
SELECT
    R.ReportID,
    R.ReportDate,
    A.AdminName,
    A.Role AS AdminRole,
    R.ReportType,
    R.Description
FROM
    Report R
JOIN
    Admin A ON R.AdminID = A.AdminID;
-- V5: High_Value_Customer_View
CREATE VIEW High_Value_Customer_View AS
SELECT
    C.CustomerID,
    C.CustomerName,
```

```
    C.Email,
    SUM(PO.TotalAmount) AS TotalSpent
FROM
    Customer C
JOIN
    PurchaseOrder PO ON C.CustomerID = PO.CustomerID
GROUP BY
    C.CustomerID, C.CustomerName, C.Email
HAVING
        SUM(PO.TotalAmount) > 5000.0;
```